Contents

Kilonova SED pipeline — user guide 1
1. Prerequisites . . . 2
1.1 Software . . . 2
1.2 The COCO_PATH environment variable . . . 2
1.3 Headless plotting . . . 2
1.4 Repository layout . . . 2
2. First complete run (AT2017gfo) . . . 3
2.1 Environment check . . . 3
2.2 Optional reset . . . 3
2.3 Step 0: magnitudes to fluxes (always run first) . . . 3
2.4 Preparation notebooks (if needed) . . . 4
2.5 Production scripts . . . 4
2.6 What each command reads and writes . . . 4
2.7 Expected artifacts checklist . . . 5
3. Input data reference . . . 5
3.1 Photometry stages . . . 5
3.2 Spectroscopy stages . . . 7
3.3 Filters . . . 7
3.4 Supernova info table . . . 8
3.5 Scale groups JSON . . . 8
4. Notebooks — guidance . . . 8
4.1 0.1_Smooth_spectra_KN.ipynb . . . 8
4.2 1_LC_DustCorrection_KN.ipynb . . . 9
4.3 2_LC_modelRising_KN_fullfit_log.ipynb . . . 9
4.4 LCfit_publication_plots.ipynb (optional) . . . 9
4.5 Downstream QA notebooks (7.5_*.ipynb, 7.6_synthetic_photometry_final_gp.ipynb, bolometric_luminosity.ipynb, plotting-final.ipynb) . . . 9
5. Script drivers . . . 9
5.0 0_Mag_to_Flux_KN.py . . . 10
5.1 3_LCfit_KN_log.py . . . 11
5.2 4_Scale_spectra_KN.py . . . 11
5.3 5_Mangle_spectra_KN_log.py . . . 12
5.4 6_Iterative_GP_mangle_KN.py . . . 13
5.5 clear_sn_run.py . . . 13
6. Iteration plot layout and quality assurance . . . 14
7. pipeline_config.py reference . . . 14
7.1 Object identity and paths . . . 14
7.2 Prescale (step 4) . . . 14
7.3 Mangling (step 5 and remangle in step 6) . . . 15
7.4 Iterative two-dimensional Gaussian Process (step 6) . . . 15
7.5 Plotting style . . . 15
8. Troubleshooting . . . 16
9. Developer documentation and PDF generation . . . 17

Kilonova SED pipeline — user guide

This document is the full instruction manual for **kn-sed-pipeline**: a standalone log-space photometry and spectroscopy pipeline that builds a time-evolving spectral energy distribution (SED) from multi-band light curves and sparse spectra. The default worked example is **AT2017gfo**.

Read this guide when you need to run the pipeline end to end, prepare a new transient, interpret outputs, or change configuration. For a shorter first-pass overview, start with the repository README. For equations

and scientific rationale, see the Pipeline Writeup.

---

1. Prerequisites

1.1 Software

You need:

- **Python 3.10 or newer** (Python 3.12 is tested in development).
- A virtual environment is strongly recommended.
- All packages in `requirements.txt`. The non-standard dependency that most often needs attention is **george** (Gaussian Process library). Also required: numpy, scipy, pandas, matplotlib, astropy, extinction, synphot, emcee, scikit-learn, and jupyter (for preparation notebooks).

Install with:

```
cd /path/to/kn-sed-pipeline
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

1.2 The `COCO_PATH` environment variable

Every notebook and script resolves paths relative to the repository root. That root is taken from the environment variable **`COCO_PATH`**, or, if unset, from the parent of the `Codes/` directory when `pipeline_config.py` is imported.

```
export COCO_PATH="/path/to/kn-sed-pipeline"
```

`COCO_PATH` must point at the folder that contains `Codes/`, `Inputs/`, and `Outputs/`. Trailing slashes are normalized by the configuration module.

1.3 Headless plotting

On servers without a display, set:

```
export MPLBACKEND=Agg
```

Production scripts already force the Agg backend where they create figures. Setting it in the shell avoids failures when you import plotting helpers from an interactive session.

1.4 Repository layout

Path	Purpose
`Codes/`	Preparation notebooks, production `.py` drivers, and `pipeline_config.py`
`Codes/helper_scripts/`	Shared libraries (light-curve fit, prescale, mangling, iterative Gaussian Process)
`Codes/helper_scripts/twodim_gp/`	Vendored two-dimensional Gaussian Process inference and plots
`Codes/helper_scripts/docs/`	Developer phase notes (prescale, iteration, data flow, QA)
`Codes/helper_scripts/tests/`	Unit tests
`Inputs/Photometry/`	Light-curve stages (`1_` raw AB mags + step-0 fluxes → `2_` dust-corrected → `3_` extrapolated)
`Inputs/Spectroscopy/`	Original, smoothed, and prescaled spectra plus list files
`Inputs/Filters/`	Filter throughput tables (`GeneralFilters/` and variants)
`Inputs/SNe_Info/`	Metadata table (`info.dat`) used by dust correction

Path	Purpose
`Inputs/Standards/`	Vega reference spectrum (`alpha_lyr_stis_006.fits`, CALSPEC) used by step 0 to convert Vega magnitudes to AB
`Inputs/2DIM_priors/`	Optional surface priors for the two-dimensional Gaussian Process
`Outputs/<SN>/`	All pipeline products for one supernova or kilonova name
`docs/`	This user guide, the scientific writeup, and PDF build scripts

---

2. First complete run (AT2017gfo)

This section walks through a full production run on the shipped AT2017gfo inputs. Always start with step 0 (§2.3), which checks that the raw photometry fluxes are consistent with the AB magnitudes and filter curves. It then assumes notebooks 0.1–2 have produced smoothed spectra and an extrapolated light curve, or that you are willing to run those notebooks first.

2.1 Environment check

```
export COCO_PATH="/path/to/kn-sed-pipeline"
cd "$COCO_PATH/Codes"
source ../.venv/bin/activate   # if you use a venv
python -c "import pipeline_config as p; print(p.COCO_PATH); print(p.SNNAME_DEFAULT)"
```

You should see your repository path and `AT2017gfo`.

2.2 Optional reset

If a previous run left stale products:

```
python clear_sn_run.py --snname AT2017gfo --yes
```

This empties `Outputs/AT2017gfo/` but keeps the directory. Prescaled spectra under `Inputs/` are **not** deleted unless you also pass `--include-input-intermediates` (requires `--yes`). Use `--dry-run` to list targets without deleting.

2.3 Step 0: magnitudes to fluxes (always run first)

```
cd "$COCO_PATH/Codes"
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --check
```

`--check` recomputes `Flux`/`Flux_err` in `Inputs/Photometry/1_LCs_flux_raw/AT2017gfo.dat` from `Mag`/`Mag_err` at each filter's pivot wavelength (§5.0) and prints a per-band table. It exits with status 0 if the file is consistent and 1 if any band is stale; it never writes.

If it fails, or you have new photometry, rebuild the raw file. **Rename the old file first** — step 0 refuses to replace an existing file unless you pass `--overwrite`:

```
mv ../Inputs/Photometry/1_LCs_flux_raw/AT2017gfo.dat ../Inputs/Photometry/1_LCs_flux_raw/AT2017gfo_old.
# either: build it from the compiled-photometry JSON (§3.1)
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --from-json /path/to/KN_SED_DATA_dust_corrected.json
# or: keep the old rows and recompute only Flux/Flux_err
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --redo-fluxes --input ../Inputs/Photometry/1_LCs_flux_raw/
```

Whenever step 0 rewrites the raw file, every downstream product is stale. Re-run notebooks 1 and 2 (to regenerate `2_LCs_dust_corrected/` and `3_LCs_extrapolated/`) and then steps 3–6. Re-run step 0 whenever you add photometry or change a filter curve.

AT2017gfo note (October 2026). The raw `Flux` column was regenerated with step 0 (issue #3). The previous version used a blue-shifted "effective wavelength" and was 0.07–0.6 mag too bright depending on band. The shipped `2_LCs_dust_corrected/` and `3_LCs_*` files and any existing `Outputs/` were built from the old fluxes and must be regenerated (notebooks 1–2, then steps 3–6) before results are used.

2.4 Preparation notebooks (if needed)

If `Inputs/Photometry/3_LCs_extrapolated/AT2017gfo.dat` (built from the current step-0 fluxes) and `Inputs/Spectroscopy/2_spec_lists_smoothed/AT2017gfo.list` already exist, you may skip to the scripts. Otherwise open Jupyter from `Codes/` and run:

1. `0.1_Smooth_spectra_KN.ipynb`
2. `1_LC_DustCorrection_KN.ipynb`
3. `2_LC_modelRising_KN_fullfit_log.ipynb`

Confirm that `SN_EXPLOSION_MJD["AT2017gfo"]` in `pipeline_config.py` matches the explosion (merger) time used in notebook 2.

2.5 Production scripts

```
cd "$COCO_PATH/Codes"

python 3_LCfit_KN_log.py --snname AT2017gfo

python 4_Scale_spectra_KN.py --snname AT2017gfo --write-groups-only
# Open Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json and confirm
# XSHOOTER UVB/VIS/NIR groups and merge_order before continuing.
python 4_Scale_spectra_KN.py --snname AT2017gfo

python 5_Mangle_spectra_KN_log.py --snname AT2017gfo --save-epoch-plots

python 6_Iterative_GP_mangle_KN.py --snname AT2017gfo --max-iters 20 --verbose
```

2.6 What each command reads and writes

Step	Reads	Writes
0	`--from-json`: compiled-photometry JSON; `--redo-fluxes`/`--check`: `Inputs/Photometry/1_LCs_flux_raw/AT2017gfo.dat` (or `--input`); always `Inputs/Filters/<FilterSet>/<band>.dat`	`Inputs/Photometry/1_LCs_flux_raw/AT2017gfo.dat` (or `--output`; refuses to replace an existing file without `--overwrite`), `Outputs/AT2017gfo/AT2017gfo_step0_json_rows.csv` (JSON mode); nothing with `--check`
3	`Inputs/Photometry/3_LCs_extrapolated/AT2017gfo.dat`, spectrum list (for spectroscopy Modified Julian Dates)	`Outputs/AT2017gfo/fitted_phot4mangling_AT2017gfo.dat`, `fitted_phot_logspace_AT2017gfo.dat`, optional `fittedGP_*.pdf`
4 (groups only)	`2_spec_lists_smoothed/AT2017gfo.list`	`Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json`
4 (scale)	Smoothed spectra + groups JSON	`Inputs/Spectroscopy/2_spec_prescaled/AT2017gfo/`, `2_spec_lists_prescaled/AT2017gfo.list`, scale report and diagnostics
5	Prescaled list + spectra, `fitted_phot4mangling_*.dat`, filters, raw photometry for band Modified Julian Date windows	`Outputs/AT2017gfo/mangled_spectra/`, `AT2017gfo_band_mjd_ranges.json` (if missing), `figs/mangle_*`

Step	Reads	Writes
6	Mangled spectra, both fitted photometry tables, prescaled list, groups JSON	`Outputs/AT2017gfo/twodim_iter/` (iterations, metrics, final products, diagnostics)

2.7 Expected artifacts checklist

Step	Key outputs to verify
0	`python 0_Mag_to_Flux_KN.py --snname AT2017gfo --check` prints `OK` and exits 0
3	`fitted_phot4mangling_AT2017gfo.dat` and `fitted_phot_logspace_AT2017gfo.dat` exist and are non-empty
4	Prescaled directory contains one file per smoothed spectrum; groups JSON lists XSHOOTER triplets
5	`mangled_spectra/` has one mangled file per successfully constrained epoch; epoch PDFs under `figs/mangle_epoch/`
6	`twodim_iter/final/full_gp/` has extended spectra; `twodim_iter/diagnostics_summary/index.html` opens; `gp_vs_mangled` PDFs show full instrument wavelength spans

Runtime for step 6 depends on iteration count and machine; a short debug run with `--max-iters 2` is useful before a full `--max-iters 20` production pass.

---

3. Input data reference

3.1 Photometry stages

Stage folder	Written by	Role
`Inputs/Photometry/1_LCs_flux_raw/`	Step 0 (from a compiled-photometry JSON, or `--redo-fluxes` on a user-written file)	Observed (not dust-corrected) AB magnitudes and their pivot-wavelength band fluxes; also used to auto-build `<SN>_band_mjd_ranges.json` in step 5
`Inputs/Photometry/2_LCs_dust_corrected/`	Notebook 1	Dust-corrected fluxes
`Inputs/Photometry/3_LCs_extrapolated/`	Notebook 2	Input to step 3 light-curve Gaussian Process
`Inputs/Photometry/3_LCs_early_extrapolated/`	Notebook 2 (intermediate)	Early-only preview or intermediate products in some workflows

Legacy checkouts may still have `4_LCs_late_extrapolated/`. The configuration module falls back to that

path for the extrapolated stage if `3_LCs_extrapolated/` is missing, and emits a warning. Prefer migrating files to `3_LCs_extrapolated/`.

Filename: always `<SN>.dat` (example: `AT2017gfo.dat`).

Extrapolated columns (typical): `MJD`, `band`, `Flux`, `Flux_err`, `FilterSet`, `Instr`, `Phase`, `Log_Phase`, `Log_Flux`, `Log_Flux_err`.

Raw columns (required): `MJD`, `Mag`, `Mag_err`, `Flux`, `Flux_err`, `band`, `Instr`, `FilterSet` (comma-separated). `Mag` is an **AB** magnitude in the filter system the measurement was reported in; `band` must match `Inputs/Filters/<FilterSet>/<band>.dat`. Step 0 always writes `Flux`/`Flux_err`. An optional `MagSys` column is allowed but must be `AB`.

Compiled-photometry JSON (input to `--from-json`). A JSON list with one record per observation, as exported from the photometry compilation spreadsheet (`KN_SED_Data.json`, `KN_SED_DATA_dust_corrected.json`). Keys used:

Key	Use
`MJD`	Copied to `MJD` (must be numeric)
`Telescope`, `Instrument`, `Filter`	Matched to a filter curve: `<Instrument>_<Filter>`, else `<Telescope>_<Filter>`; exact, then case-insensitive. `band` is the matched file name; `Instr` is `Instrument` (or `Telescope` if `Instrument` is `-`)
`Villar Mag`, `Villar Mag_err`	Copied to `Mag`, `Mag_err` exactly as given (override the keys with `--mag-key`, `--mag-err-key`)
`AB/VEGA/Etc`	`AB` kept; `Vega` **converted to AB** (below); `-`/empty kept **as AB with a warning**; anything else dropped
`Used in Villar model?`	Only with `--villar-used-only`: keep rows marked `*`
`Repeated obs`	Only with `--drop-repeated`: drop rows marked `R` (repeated observation), keep everything else
`Refrence`	Copied to the row report

Magnitudes must **not** be dust-corrected (notebook 1 applies the extinction correction), so any published dereddening has to be undone in the JSON before step 0; step 0 cannot tell from the JSON whether that was done. Rows are dropped when MJD, magnitude, or error is non-numeric, the magnitude is a limit (`>`/`<`), the system is neither AB nor Vega, or no filter curve matches (add the curve under `Inputs/Filters/GeneralFilters/` to keep them).

Vega magnitudes. A Vega magnitude is defined like an AB magnitude but relative to the spectrum of Vega (Vega = 0 mag in every band). Because both use the same photon-weighted band mean, each filter has an exact, SED-independent offset

$$m_{AB} = m_{\mathrm{Vega}} - 2.5 \log_{10} \frac{\int f_\lambda^{\mathrm{Vega}} S\lambda \, d\lambda}{\int f_\lambda^{AB} S\lambda \, d\lambda},$$

computed from `Inputs/Standards/alpha_lyr_stis_006.fits` (override with `--vega-spectrum`). Step 0 writes the converted AB magnitude to `Mag` and records the original magnitude, system, and offset in the row report (`input_mag`, `input_system`, `Mag_AB`, `reason`). Typical offsets: about -0.01 in V, -0.10 in B, $+0.4$ in i, $+0.92$ in J, $+1.83$ in K_s.

Repeated observations. The compilation often lists the same measurement from several papers; Villar et al. mark the copies with `R` in `Repeated obs` and use one of them (often their own averaged value, `A`). `--drop-repeated` removes the `R` rows only, so rows Villar excluded for other reasons stay in; `--villar-used-only` applies Villar's full selection instead. Rows that repeat an MJD and band (the

same epoch from several papers) are kept but flagged. A repeated CSV header record is skipped. Every decision is written to `Outputs/<SN>/<SN>_step0_json_rows.csv` (`status` = `kept` / `kept_assumed_AB` / `kept_converted_from_Vega` / `dropped`, plus `reason`).

Flux convention. Every `Flux` in the pipeline is the photon-weighted band-mean F_λ in erg s^{-1} cm^{-2} Å^{-1},

$$\langle f_\lambda \rangle = \frac{\int f_\lambda S\lambda \, d\lambda}{\int S\lambda \, d\lambda} = 3631\,\mathrm{Jy} \times 10^{-0.4 m_{AB}} \, \frac{c}{\lambda_{\mathrm{piv}}^2}, \qquad \lambda_{\mathrm{piv}} = \sqrt{\frac{\int S\lambda \, d\lambda}{\int S/\lambda \, d\lambda}},$$

where $S(\lambda)$ is the filter's photon-counting throughput. The second equality holds exactly for AB magnitudes, independent of the source SED, so step 0 converts at the **pivot** wavelength. This is the same quantity the mangling step fits (`band_flux_trapz`) and the standard (photon-counting) AB definition used for synthetic photometry. Errors propagate as $\sigma_F = F \cdot 0.4 \ln 10 \cdot \sigma_m$, which is adequate for S/N $\gtrsim 10$. Log columns use $\log_{10}$ flux.

3.2 Spectroscopy stages

Folder	Role
`Inputs/Spectroscopy/1_spec_original/<SN>/`	Unsmoothed spectrum files
`Inputs/Spectroscopy/1_spec_lists_original/`	Original list files (`<SN>.list`)
`Inputs/Spectroscopy/2_spec_smoothed/<SN>/`	Smoothed spectra (notebook 0.1 or user-provided)
`Inputs/Spectroscopy/2_spec_lists_smoothed/`	List pointing at smoothed files
`Inputs/Spectroscopy/2_spec_prescaled/<SN>/`	Flux-rescaled arms after step 4
`Inputs/Spectroscopy/2_spec_lists_prescaled/`	List pointing at prescaled files (used by steps 5–6 when `USE_PRESCALED_SPECTRA=True`)

List format: three whitespace-separated fields per line — Modified Julian Date, phase (days), path to spectrum file. Paths may be absolute; the prescale helpers rewrite stale roots that still contain older repository names when possible.

Spectrum file format: ASCII with wavelength (angstroms), flux, and flux error. A header line starting with `#` is allowed:

```
#wls    flux    flux_err
3725.6000   1.8021e-15  4.1786e-16
```

Naming conventions that matter for grouping:

- XSHOOTER arms: filenames containing `UVB_`, `VIS_`, `NIR_` (or similar) near the same Modified Julian Date are grouped together.
- Single-instrument spectra (for example Magellan/LDSS3) remain ungrouped individuals unless you edit the groups JSON.

3.3 Filters

Throughput tables live under `Inputs/Filters/GeneralFilters/` for the default AT2017gfo path: two columns, wavelength in Å and **photon-counting** throughput (filter × detector QE × optics; the overall normalization does not matter). The mangling code and step 0 resolve each band name to a transmission file. Use the curve of the system the magnitudes are **reported** in: the instrument's natural system for natural-system photometry, or the standard system's curve if a paper transformed its photometry to a standard system. If you add or replace a filter curve, re-run step 0, because the pivot wavelength changes. Events that need Carnegie Supernova Project (CSP) natural-system filters can be listed in `CSP_SNE` inside `pipeline_config.py` so that CSP-specific filter directories are preferred.

3.4 Supernova info table

`Inputs/SNe_Info/info.dat` holds coordinates and type fields used by the dust-correction notebook (Milky Way extinction toward the transient). Add one row per new object following the header comments in that file.

3.5 Scale groups JSON

Step 4 writes `Outputs/<SN>/<SN>_spec_scale_groups.json`. A minimal structure looks like:

```
{
  "default_output_mode": "scale_only",
  "groups": [
    {
      "id": "group_suggested_000",
      "members": [
        "NIR_AT2017gfo_..._MJD-57983.969001_....dat",
        "UVB_AT2017gfo_..._MJD-57983.969002_....dat",
        "VIS_AT2017gfo_..._MJD-57983.969003_....dat"
      ],
      "merge_order": ["uvb", "vis", "nir"],
      "output_mode": "scale_only",
      "reason": "same_time_cluster"
    }
  ],
  "ungrouped": [
    "57983.01825_LDSS3_Magellan.dat"
  ]
}
```

- **`members`:** basenames of spectrum files that share a flux scale.
- **`merge_order`:** arm order for bridging and optional merging (`uvb`, `vis`, `nir`).
- **`output_mode`:** `scale_only` keeps separate files (default); `merge_join` writes one stitched spectrum per group.
- **`ungrouped`:** spectra scaled only to themselves (factor 1.0) and mangled individually.

Edit this file carefully after `--write-groups-only`. Incorrect groups produce seam jumps and poor bundle mangling.

---

4. Notebooks — guidance

Each notebook should call into `pipeline_config` (or `bootstrap_runtime`) rather than hard-coding absolute paths. Set `snname` to your object and ensure `COCO_PATH` is exported before launching Jupyter.

4.1 `0.1_Smooth_spectra_KN.ipynb`

Purpose. Reduce noise in raw spectra while preserving broad continuum shape needed for later synthetic photometry.

Typical cell order. Import and paths → choose smoothing parameters → loop over the original list → write smoothed files and an updated list under `2_spec_smoothed/` and `2_spec_lists_smoothed/`.

Outputs. Smoothed ASCII spectra and `<SN>.list` in the smoothed folders.

Common failures. Missing original list path; spectrum files with wrong column order; writing to a path outside `COCO_PATH` so later steps cannot find the list.

4.2 `1_LC_DustCorrection_KN.ipynb`

Prerequisite. Step 0 (`0_Mag_to_Flux_KN.py --check`) passes, so the raw `Flux` column is current.

Purpose. Correct photometry for Milky Way (and optional host) extinction so fluxes are closer to intrinsic values before light-curve fitting.

Typical cell order. Load `info.dat` and raw light curve → compute A_λ from extinction law and $E(B-V)$ → write dust-corrected light curve.

Outputs. `Inputs/Photometry/2_LCs_dust_corrected/<SN>.dat`.

Common failures. Missing coordinates in `info.dat`; filter names in the light curve that do not match filter tables.

4.3 `2_LC_modelRising_KN_fullfit_log.ipynb`

Purpose. Model the early rising light curve (Bazin and related functional forms), optionally force zero flux at explosion, extrapolate late times where needed, and commit the explosion Modified Julian Date used as phase zero downstream.

Typical cell order. Load dust-corrected photometry → preview rising fits per band → commit extrapolation → write `3_LCs_extrapolated/<SN>.dat` → update or confirm `SN_EXPLOSION_MJD` in configuration.

Outputs. Extrapolated light curve under `Inputs/Photometry/3_LCs_extrapolated/`; possible intermediate early-extrapolation products.

Common failures. Explosion time inconsistent with configuration; excluding too many bands so some filters never get a rising model.

4.4 `LCfit_publication_plots.ipynb` (optional)

Purpose. Publication-quality light-curve figures after step 3 has written the Gaussian Process fit products.

When to run. After `3_LCfit_KN_log.py` succeeds.

4.5 Downstream QA notebooks (`7.5_*.ipynb`, `7.6_synthetic_photometry_final_gp.ipynb`, `bolometric_luminosity.ipynb`, `plotting-final.ipynb`)

Purpose. Compare synthetic photometry from final spectra to the light-curve fit; overlay spectra; construct bolometric luminosity; assemble final figures. `7.6_synthetic_photometry_final_gp.ipynb` writes synthetic AB magnitudes of `FINAL_spectra_2dim/as_observed` through every observed filter, both on the GP epochs and at each input-photometry row, for chi-squared comparisons against the input data. Use its default `SYNTH_METHOD = "true_ab"`, which matches the step-0 convention.

Configuration. Keep `USE_ITER_GP_MANGLE_FINAL = True` (default) so path helpers resolve `twodim_iter/final/` and the QA export under `FINAL_spectra_2dim/`. Re-run the configuration and data-directory cells after changing that flag.

Prerequisites. A finished step 6 run with `final/full_gp/` populated (and QA export enabled if you rely on `_FINAL_spec` naming).

Common failures. Pointing at empty legacy directories because the iteration flag is off; looking for calendar Modified Julian Date stems when files are keyed by log-phase stems from the Gaussian Process grid.

---

5. Script drivers

All drivers live in `Codes/` and accept `--snname` and usually `--coco-path`. Defaults come from `pipeline_config.py`.

5.0 `0_Mag_to_Flux_KN.py`

Science role. Creates or updates `Inputs/Photometry/1_LCs_flux_raw/<SN>.dat`, converting AB magnitudes into the band-mean F_λ used everywhere downstream (§3.1). Exactly one mode is required:

Mode	What it does
`--from-json PATH`	Builds the raw file from a compiled-photometry JSON (§3.1): filter matching, row selection, flux conversion, row report
`--redo-fluxes`	Reads an existing raw-format file (default: the raw file; or `--input`) and recomputes only `Flux`/`Flux_err`; all other columns unchanged
`--check`	Verifies an existing raw-format file; exit 1 if any band's `Flux` is stale; never writes

The conversion is $F = 3631\,\mathrm{Jy} \times 10^{-0.4\,\mathrm{Mag}}\, c/\lambda_{\mathrm{piv}}^2$, with λ_{piv} computed from `Inputs/Filters/<FilterSet>/<band>.dat`, and $\sigma_F = F \cdot 0.4 \ln 10 \cdot \sigma_{\mathrm{Mag}}$. With `--redo-fluxes`, every other column is left unchanged and rows with an empty `Mag` keep their existing `Flux`.

Flags.

Flag	Default	Meaning
`--snname`	`SNNAME_DEFAULT`	Object name (`<SN>.dat`)
`--coco-path`	`COCO_PATH`	Repository root
`--input`	raw file	Raw-format file to read for `--redo-fluxes` / `--check`
`--output`	raw file	Where to write (`--from-json`, `--redo-fluxes`)
`--overwrite`	off	Allow replacing an existing output file. **Rename the old file first**
`--filter-set`	`GeneralFilters`	Filter folder under `Inputs/Filters/` for `--from-json`
`--villar-used-only`	off	`--from-json`: keep only rows marked `*` in `Used in Villar model?`
`--drop-repeated`	off	`--from-json`: drop rows marked `R` in `Repeated obs`; keep everything else
`--vega-spectrum`	`Inputs/Standards/alpha_lyr_stis_006.fits`	`--from-json`: Vega spectrum used to convert Vega magnitudes to AB
`--mag-key`, `--mag-err-key`	`Villar Mag`, `Villar Mag_err`	`--from-json`: JSON keys holding the AB magnitude and its error
`--report`	`Outputs/<SN>/<SN>_step0_json_rows.csv`	`--from-json`: per-row report path
`--rtol`	`1e-6`	Relative tolerance for `--check`

Output. `--from-json` prints how many records were kept/dropped (with reasons), the rows kept with warnings (assumed AB, converted from Vega with the offset used, case-insensitive filter match), a warning if epochs are repeated, and the pivot wavelength and row count per band; it writes the raw file and the row report. `--redo-fluxes` / `--check` print a per-band table (pivot wavelength, number of rows, median `-2.5 log10(Flux_old/Flux_new)`, maximum relative change) followed by the path written or `OK`/`FAIL`.

> **Back up before writing.** `--from-json` and `--redo-fluxes` write `Inputs/Photometry/1_LCs_flux_raw/<SN>.dat` by default. If that file already exists the script **refuses to run** unless you pass `--overwrite`. **Rename the old file first** (e.g. `mv AT2017gfo.dat AT2017gfo_old.dat`) so nothing is lost, or write elsewhere with `--output`.

```
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --from-json /path/to/KN_SED_DATA_dust_corrected.json
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --redo-fluxes --input old_AT2017gfo.dat
python 0_Mag_to_Flux_KN.py --snname AT2017gfo --check
```

Common failures. Exit status 2 with "Refusing to overwrite" (the output exists: rename it or pass `--overwrite`); many rows dropped with "no filter curve" (add the curve or check the JSON `Instrument/Telescope/Filter` spelling against the file names); `FileNotFoundError` for a band in `--redo-fluxes`/`--check` (the `band` value does not match a filter file name); `ValueError` for non-AB `MagSys` values (only AB is supported).

5.1 3_LCfit_KN_log.py

Science role. For each photometric band, fit a one-dimensional Gaussian Process to $\log_{10}$ flux versus time (or log phase). Evaluate the posterior at every spectroscopy Modified Julian Date (for mangling constraints) and on a dense log-phase grid (for the two-dimensional Gaussian Process training table).

Implementation. `helper_scripts/lc_gp_fit.py` with kernels from `lc_gp_kernels.py`.

Inputs. Extrapolated light curve; spectrum list for epoch list; optional explosion anchor.

Outputs. `fitted_phot4mangling_<SN>.dat`, `fitted_phot_logspace_<SN>.dat`, optional summary PDFs.

Flag	Default	Effect
`--snname`	`SNNAME_DEFAULT`	Target object
`--coco-path`	`COCO_PATH`	Repository root
`--kernel-settings`	None	Per-band kernel JSON overrides
`--save-general-plots` / `--no-save-general-plots`	on	Summary `fittedGP_*.pdf`
`--save-per-filter-plots`	off	One PDF per filter under `lc_gp_per_filter/`
`--anchor-t0-in-lc-gp`	config	Include explosion-time anchor in training
`--append-t0-row`	off	Extra row in the logspace photometry file
`--exclude-filt`	None	Override band exclusion list

Worked examples.

```
# Standard production
python 3_LCfit_KN_log.py --snname AT2017gfo

# Per-filter diagnostic PDFs
python 3_LCfit_KN_log.py --snname AT2017gfo --save-per-filter-plots
```

5.2 4_Scale_spectra_KN.py

Science role. Multi-arm spectrographs (especially XSHOOTER) deliver separate UVB, VIS, and NIR files whose absolute flux calibrations may disagree in overlap regions. This step estimates multiplicative scale factors so arms join smoothly before mangling.

Implementation. `helper_scripts/spectra_pre_scale.py`.

Inputs. Smoothed list and spectra; optional existing groups JSON.

Outputs. Prescaled spectra and list; groups JSON; scale report; optional HTML diagnostics under `spec_scale_diagnostics/`.

Flag	Effect
`--write-groups-only`	Write or refresh the groups JSON template; do not scale
`--groups-json`	Override groups path
`--output-mode`	`scale_only` or `merge_join`
`--chain-mode`	Sequential arm bridging along `merge_order`
`--gap-max-a`	Maximum gap (angstroms) for gap-seam scaling
`--write-diagnostics`	Seam QA under `spec_scale_diagnostics/`
`--same-time-minutes`	Clustering window for suggested groups
`--verbose`	Extra logging

Worked examples.

```
# Always do this two-step dance for a new object
python 4_Scale_spectra_KN.py --snname AT2017gfo --write-groups-only
# edit Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json
python 4_Scale_spectra_KN.py --snname AT2017gfo --write-diagnostics
```

5.3 `5_Mangle_spectra_KN_log.py`

Science role. At each spectroscopy epoch, compare synthetic photometry of the (prescaled) spectrum to the step-3 light-curve fit in every overlapping band. Fit a smooth mask $m(\lambda)$ in log flux so that after mangling, band fluxes match the light curve. Bundle-aware mode fits one shared mask per scale group using stitched synthetic photometry across arms.

Implementation. helper_scripts/mangle_spectra_log.py.

Inputs. Prescaled list (default), `fitted_phot4mangling_*.dat`, filters, groups JSON, raw photometry for band Modified Julian Date ranges.

Outputs. `mangled_spectra/`; auto-written `*_band_mjd_ranges.json` when missing; `figs/mangle_nb5/` and optional `figs/mangle_epoch/`.

Flag	Effect
`--bundle-aware` / `--no-bundle-aware`	Shared mask per scale group
`--save-diagnostics`	HTML and PDF under `figs/mangle_nb5/`
`--save-epoch-plots`	Per-epoch PDFs under `figs/mangle_epoch/`
`--run-both-for-diag`	Per-arm and bundle compare (diagnostics only)
`--mangle-kernel-mode`	`fixed_5` (PyCoCo parity, default) or `kernel_divide_scaled`
`--kernel-divide`	Numerator when mode is `kernel_divide_scaled` (default 800)
`--groups-json`	Override groups path
`--verbose`	Extra logging

Worked examples.

```
# Production with epoch plots
python 5_Mangle_spectra_KN_log.py --snname AT2017gfo --save-epoch-plots

# Compare kernel modes for diagnostics
python 5_Mangle_spectra_KN_log.py --snname AT2017gfo --mangle-kernel-mode fixed_5
python 5_Mangle_spectra_KN_log.py --snname AT2017gfo --mangle-kernel-mode kernel_divide_scaled --kernel
```

If `Outputs/<SN>/<SN>_band_mjd_ranges.json` is missing, step 5 builds it from `Inputs/Photometry/1_LCs_flux_raw/<SN>.`
You do not need legacy repository-root Modified Julian Date CSV files.

5.4 6_Iterative_GP_mangle_KN.py

Science role. Outer loop: fit a two-dimensional Gaussian Process on mangled spectra plus photometry; extract the surface at observed spectroscopy epochs; remove the current mangling mask (demangle); build a new mask from demangled Gaussian Process spectra versus the light-curve fit (often on the full Gaussian Process wavelength grid); apply the new mask to **prescaled originals**; repeat until photometry closure or the iteration cap.

Implementation. helper_scripts/iterative_gp_mangle.py, with Gaussian Process fitting via iter_gp_grid.py and the vendored twodim_gp/ package.

Inputs. Step 5 mangled spectra (seed for iter_00), both fitted photometry tables, prescaled list and files, groups JSON, filters.

Outputs. Outputs/<SN>/twodim_iter/ including per-iteration mangled spectra, predictions, figures, metrics, and final/.

Flag	Effect
--max-iters	Cap (config default 20)
--phot-convergence-frac	Stop when max relative photometry error is below this
--bundle-aware	Bundle remangle
--seed-from-nb5	Copy step-5 mangled spectra into iter_00
--warm-start	Warm-start Gaussian Process hyperparameters from the previous iteration
--save-diagnostics	Write figure suites and metrics
--t0-fix	Override explosion Modified Julian Date
--verbose	Extra logging

Worked examples.

```
# Short debug
python 6_Iterative_GP_mangle_KN.py --snname AT2017gfo --max-iters 2 --verbose

# Production
python 6_Iterative_GP_mangle_KN.py --snname AT2017gfo --max-iters 20 --verbose
```

Default ITER_MANGLE_USE_GP_WAVELENGTH_GRID=True means remangle constraints are built from a demangled Gaussian Process SED sampled across the **full training wavelength range**, not only each arm's native span. The new mask is still applied to the original prescaled wavelength grids.

5.5 clear_sn_run.py

Role. Delete or list output products for one supernova so you can re-run cleanly.

Flag	Effect
--snname	Required object name
--dry-run	Print paths only
(default)	Empty Outputs/<SN>/ but keep the directory
--include-input-intermediates	Also delete prescaled spectroscopy directory and list (**requires --yes**)
--yes	Confirm destructive actions

Warning. --include-input-intermediates removes Inputs/Spectroscopy/2_spec_prescaled/<SN>/ and the matching list. Re-run step 4 afterward.

6. Iteration plot layout and quality assurance

Per-iteration figures land under `Outputs/<SN>/twodim_iter/iter_KK/figs/`.

Subfolder	Content
`gp_surface/`	Heatmaps of the two-dimensional Gaussian Process mean and uncertainty, wavelength and phase slices, training residuals and coverage
`gp_vs_mangled/`	Input mangled spectrum versus Gaussian Process extract. For individuals, the x-axis is the **full prescaled wavelength span**; the Gaussian Process curve is interpolated onto that grid. Groups are stitched before plotting
`mangle_delta/`	Difference between the new mangling and the input mangling for that iteration
`residuals/`	Gaussian Process minus mangled, compared to uncertainties, on the full mangled wavelength span
`phot_lc/`	Posterior light curves and mangled synthetic photometry versus `gp_fit` targets
`debug/bundle_scaling/`	Optional bundle-pair scaler plots when `PLOT_BUNDLE_SCALING_QA=True`

Run-level metrics live in `twodim_iter/metrics/` (for example `chi2_vs_iter.png`, `lengthscales_vs_iter.png`). Gaussian Process inference caches (not plots) live under `iter_KK/gp_runs/inference/`. Open `twodim_iter/diagnostics_summary/index.html` for a clickable overview.

Detailed physical data flow for each remangle iteration is documented in Codes/helper_scripts/docs/ITERATIVE_MANGLING_D

---

7. `pipeline_config.py` reference

Edit Codes/pipeline_config.py or override via command-line flags where supported. Change a knob only when you understand which stage reads it.

7.1 Object identity and paths

Constant	Default	When to change
`SNNAME_DEFAULT`	`AT2017gfo`	Default `--snname` for scripts
`SN_EXPLOSION_MJD`	per-object dict	Phase zero / t_0; must match notebook 2
`SN_REDSHIFT`	per-object dict	Distance and rest-frame conversions where used
`CSP_SNE`	`()`	Add names that need CSP filter routing
`COCO_PATH`	env or auto	Override repository root

7.2 Prescale (step 4)

Constant	Default	When to change
`SPEC_SCALE_OUTPUT_MODE`	`scale_only`	Use `merge_join` only if you want one file per group
`SPEC_SCALE_CHAIN_MODE`	`False`	`True` for sequential bridging along `merge_order`

Constant	Default	When to change
SPEC_SCALE_STAR_BRIDGE_FALLBACK	True	One-hop bridge when the arm is not adjacent to the anchor
SPEC_SCALE_GAP_MAX_A	400	Maximum gap (angstroms) allowed for gap-seam scaling
SPEC_SCALE_SAME_TIME_MINUTES	45	Clustering window for suggested groups

7.3 Mangling (step 5 and remangle in step 6)

Constant	Default	When to change
USE_PRESCALED_SPECTRA	True	Leave on for production; off only for legacy smoothed-only tests
MANGLE_BUNDLE_AWARE	True	Shared mask per group (recommended for XSHOOTER)
MANGLE_GP_KERNEL_MODE	fixed_5	PyCoCo parity; switch to kernel_divide_scaled only for experiments
MANGLE_GP_KERNEL_FIXED	5.0	Fixed Matérn 3/2 length in normalized log-wavelength when mode is fixed_5
MANGLE_KERNEL_DIVIDE	800	Used only in kernel_divide_scaled mode
MANGLE_BUNDLE_STITCH_SYNPHOT	True	Integrate filters on stitched multi-arm spectra
MANGLE_RUN_BOTH_FOR_DIAG	False	Turn on to compare per-arm versus bundle once

7.4 Iterative two-dimensional Gaussian Process (step 6)

Constant	Default	When to change
ITER_GP_MANGLE_MAX_ITERS	20	Lower for debugging; raise only if convergence is slow but improving
PHOT_CONVERGENCE_FRAC	0.05	Primary stop: max relative synthetic photometry error
MASK_CONVERGENCE_EPS	1e-4	Secondary logging on mask change
ITER_MANGLE_USE_GP_WAVELENGTH_GRID	True	Full-wavelength remangle constraints (recommended)
ITER_GP_SEED_FROM_NB5	True	Seed iter_00 from step 5 mangled spectra
ITER_GP_WARM_START	True	Reuse hyperparameters between iterations
ITER_GP_EXPORT_FULL_GP	True	Write extended spectra under full_gp/
ITER_EXPORT_FINAL_SPEC_FOR_QA	True	Copy QA-named spectra for notebooks 7.5
USE_ITER_GP_MANGLE_FINAL	True	Notebooks 7.5 read iteration final products
ITER_GP_DIAG_FIGS	True	Write Gaussian Process surface diagnostic figures
PLOT_BUNDLE_SCALING_QA	False	Extra debug figures for bundle scaling

Two-dimensional inference hyperparameters (kernel family, additive scales, jitter floors, early-time constraints) live in GP_INFERENCE_KWARGS. Treat those as an advanced surface; defaults match the collaborator production configuration described in the writeup.

7.5 Plotting style

Band colors, markers, and exclusion lists are loaded from Codes/filter_plot_config.json. Edit that JSON when a band should be omitted from fits or drawn in a specific color.

Full constant list: see the source file and its grouped sections.

---

8. Troubleshooting

Symptom	Likely cause	What to do
`0_Mag_to_Flux_KN.py --check` fails	Raw `Flux` was made with a different conversion (or a filter curve changed)	Rename the raw file, run step 0 with `--from-json` or `--redo-fluxes --input <renamed file>`, then re-run notebooks 1–2 and steps 3–6
Step 0 exits with "Refusing to overwrite" (status 2)	The output file already exists	Rename/back up the old file (e.g. `AT2017gfo_old.dat`) and re-run, or pass `--overwrite`
Step 0 `--from-json` drops many rows with "no filter curve"	No `Inputs/Filters/GeneralFilters/<Instrument>_<Filter>.dat` or `<Telescope>_<Filter>.dat`	Add the filter curve (named to match), or fix the spelling in the JSON; see `Outputs/<SN>/<SN>_step0_json_rows.csv`
Synthetic mags of the final SED are systematically brighter or fainter than the catalog `Mag` (by a band-dependent amount)	Pipeline was run on fluxes not produced by step 0, or `SYNTH_METHOD` is not `true_ab`	Run step 0 `--check`; regenerate downstream products; use `true_ab` in notebook 7.6
`no_phot_constraints` in step 5	No light-curve-fit bands overlap the spectrum epoch within band Modified Julian Date windows	Check `*_band_mjd_ranges.json`, band names, and whether step 3 wrote finite fits for those bands
Missing `*_band_mjd_ranges.json`	Outputs were cleared and step 5 has not run yet	Re-run step 5; it regenerates the JSON from `1_LCs_flux_raw`
Empty or short prescaled list	Step 4 not run, or smoothed list empty	Re-run notebook 0.1 and step 4; verify list paths resolve under `COCO_PATH`
Wild oscillations in mangling mask	Kernel mode not `fixed_5`, or too few constraints	Prefer `--mangle-kernel-mode fixed_5`; inspect epoch plots and used filters
`n_skipped_extract` in step 6	Epoch was not mangled in step 5	Expected for epochs without photometry constraints; fix step 5 if the epoch should have been mangled
Empty bundle epoch plot	No photometry constraints on the reference arm	Inspect bundle diagnostics; confirm groups and filter overlap
Missing `figs/gp_surface`	Gaussian Process fit failed or `ITER_GP_DIAG_FIGS=False`	Check iteration logs; enable diagnostics; verify `predictions.npz` exists
`gp_vs_mangled` shows only ~200 Å	Outdated plotting bug (fixed): truncating prescaled arrays to Gaussian Process grid length	Update `iter_plot_suite.py` and regenerate figures
Iteration never converges	Photometry targets inconsistent, or mask unstable	Inspect `phot_lc/` and `metrics/`; try warm-start on; verify step 3 and step 5 quality first

Notebooks 7.5 find no spectra	`USE_ITER_GP_MANGLE_FINAL` false or `final/full_gp/` empty	Set the flag true; confirm step 6 export flags; re-run config cells

Run the unit tests after significant code or environment changes:

```
cd Codes
PYTHONPATH=.:helper_scripts python -m unittest discover -s helper_scripts/tests -v
```

---

9. Developer documentation and PDF generation

Internal phase notes (still useful for contributors):

- Prescale
- Iterative Gaussian Process + mangling
- Physical data flow
- Downstream QA
- Deprecated modules

Scientific equations for the whole pipeline: PIPELINE_WRITEUP.md.

Build PDFs (requires pandoc; the writeup also needs a TeX distribution providing `pdflatex`):

```
docs/build_user_guide_pdf.sh
docs/build_writeup_pdf.sh
```